==========
FILE 1 — chaos/chaosEngine.js
==========

```
/* chaosEngine.js */
class ChaosEngine {
  constructor(bus) {
    this.bus = bus;
    this.state = { turbulence: 0.5, tick: 0 };
  }
  tick() {
    this.state.tick++;
    this.state.turbulence =
      Math.abs(Math.sin(this.state.tick * 0.17)) * 0.9 + 0.1;
    this.bus.emit("CHAOS_UPDATE", { turbulence: this.state.turbulence });
  }
}
export default ChaosEngine;
```

==========
FILE 2 — chaos/chaosNoise.js
==========

```
/* chaosNoise.js */
export function chaosNoise(n) {
  return (Math.sin(n * 12.9898) * 43758.5453) % 1;
}
```

==========
FILE 3 — emotion/emotionSwarm.js
==========

```
/* emotionSwarm.js */
class EmotionSwarm {
  constructor(bus) {
    this.bus = bus;
    this.state = { pressure: 0.3, tick: 0 };
  }
  tick() {
    this.state.tick++;
    this.state.pressure =
      Math.abs(Math.sin(this.state.tick * 0.11)) * 0.7 + 0.1;
    this.bus.emit("EMOTION_UPDATE", { pressure: this.state.pressure });
  }
}
export default EmotionSwarm;
```

==========
FILE 4 — emotion/emotionWeights.js
==========

```
/* emotionWeights.js */
export const emotionWeights = {
   joy: 0.5,
   fear: 0.4,
   drive: 0.6,
   stress: 0.7
};
```

==========
FILE 5 — resonance/resonanceNetwork.js
==========

```
/* resonanceNetwork.js */
class ResonanceNetwork {
   constructor(bus) {
      this.bus = bus;
      this.state = { coherence: 0.6, tick: 0 };
   }
   tick() {
      this.state.tick++;
      this.state.coherence =
         (Math.sin(this.state.tick * 0.07) + 1) / 2;
      this.bus.emit("RESONANCE_UPDATE", { value: this.state.coherence });
   }
}
export default ResonanceNetwork;
```

==========
FILE 6 — resonance/resonanceMatrix.js
==========

```
/* resonanceMatrix.js */
export const resonanceMatrix = [
   [0.9, 0.3, 0.5],
   [0.4, 0.8, 0.2],
   [0.6, 0.5, 0.9]
];
```

==========
FILE 7 — lobes/frontal/frontalLobe.js
==========

```
/* frontalLobe.js */
class FrontalLobe {
```

```
  constructor(bus) {
    this.bus = bus;
    this.state = { focus: 0.5, tick: 0 };
  }
  tick() {
    this.state.tick++;
    this.state.focus =
      Math.abs(Math.sin(this.state.tick * 0.09)) * 0.8 + 0.1;
  }
}
export default FrontalLobe;
```

==========
FILE 8 — lobes/temporal/temporalLobe.js
==========

```
/* temporalLobe.js */
class TemporalLobe {
  constructor(bus) {
    this.bus = bus;
    this.state = { patterns: 0.4, tick: 0 };
  }
  tick() {
    this.state.tick++;
    this.state.patterns =
      Math.abs(Math.cos(this.state.tick * 0.12)) * 0.7 + 0.15;
  }
}
export default TemporalLobe;
```

==========
FILE 9 — lobes/parietal/parietalLobe.js
==========

```
/* parietalLobe.js */
class ParietalLobe {
  constructor(bus) {
    this.bus = bus;
    this.state = { model: 0.5, tick: 0 };
  }
  tick() {
    this.state.tick++;
    this.state.model =
      Math.abs(Math.sin(this.state.tick * 0.15)) * 0.85 + 0.1;
  }
}
```

```
export default ParietalLobe;
```

==========
FILE 10 — lobes/occipital/occipitalLobe.js
==========

```
/* occipitalLobe.js */
class OccipitalLobe {
    constructor(bus) {
        this.bus = bus;
        this.state = { abstraction: 0.5, tick: 0 };
    }
    tick() {
        this.state.tick++;
        this.state.abstraction =
            Math.abs(Math.cos(this.state.tick * 0.05)) * 0.8 + 0.15;
    }
}
export default OccipitalLobe;
```

==========
FILE 11 — lobes/limbic/limbicLobe.js
==========

```
/* limbicLobe.js */
class LimbicLobe {
    constructor(bus) {
        this.bus = bus;
        this.state = { drive: 0.45, tick: 0 };
    }
    tick() {
        this.state.tick++;
        this.state.drive =
            Math.abs(Math.sin(this.state.tick * 0.13)) * 0.75 + 0.2;
    }
}
export default LimbicLobe;
```

==========
FILE 12 — memory/memoryNode.js
==========

```
/* memoryNode.js */
export default class MemoryNode {
    constructor(value) {
        this.value = value;
        this.strength = Math.random() * 0.5 + 0.5;
```

```
        this.drift = Math.random() * 0.1;
        this.links = [];
    }
}
```

==========
FILE 13 — memory/memoryMesh.js
==========

```
/* memoryMesh.js */
import MemoryNode from "./memoryNode.js";

class MemoryMesh {
    constructor(bus) {
        this.bus = bus;
        this.nodes = [];
        this.state = { recallSignal: 0.5, tick: 0 };
    }

    add(value) {
        const node = new MemoryNode(value);
        this.nodes.push(node);
        return node;
    }

    tick() {
        this.state.tick++;
        this.state.recallSignal =
            Math.abs(Math.sin(this.state.tick * 0.07)) * 0.8 + 0.2;

        this.bus.emit("MEMORY_STATE", {
            count: this.nodes.length,
            recall: this.state.recallSignal
        });
    }
}

export default MemoryMesh;
```

==========
FILE 14 — memory/memoryWeights.js
==========

```
/* memoryWeights.js */
export const memoryWeights = {
    retention: 0.85,
```

```
    linkStrength: 0.75,
    decay: 0.05
};
```

==========
FILE 15 — thoughtSynth/thoughtSynth.js
==========

```
/* thoughtSynth.js */
class ThoughtSynth {
    constructor(bus) {
        this.bus = bus;
        this.state = {
            current: null,
            tensionRelease: 0,
            tick: 0
        };
    }

    tick() {
        this.state.tick++;

        const tension =
            Math.abs(Math.sin(this.state.tick * 0.21)) * 0.6;

        this.state.current = {
            cognitiveTension: tension,
            motivationalForce: tension * 0.7,
            affect: Math.random() * 0.4 + 0.2,
            resonance: Math.random() * 0.5 + 0.25,
            orientation: "internal"
        };

        this.bus.emit("THOUGHT_PACKET", this.state.current);
    }
}

export default ThoughtSynth;
```

==========
FILE 16 — dream/dreamEngine.js
==========

```
/* dreamEngine.js */
class DreamEngine {
    constructor(bus) {
```

```
        this.bus = bus;

        this.state = {
            active: false,
            intensity: 0.5,
            tick: 0,
            completed: false
        };

        bus.on("HOMEOSTASIS_DREAM_TRIGGER", () => {
            this.state.active = true;
            this.state.completed = false;
        });
    }

    tick(memoryNodes, emotionState, resonance) {
        if (!this.state.active) return;

        this.state.tick++;
        this.state.intensity =
            Math.abs(Math.sin(this.state.tick * 0.18)) * 1.2;

        // memory cleanup
        memoryNodes.forEach((n) => {
            n.strength *= 0.97;
            n.drift *= 0.9;
        });

        if (this.state.tick % 50 === 0) {
            this.state.active = false;
            this.state.completed = true;
        }

        this.bus.emit("DREAM_STATE", { ...this.state });
    }
}

export default DreamEngine;
```

==========
FILE 17 — dream/dreamWeights.js
==========

```
/* dreamWeights.js */
export const dreamWeights = {
```

```
    cleanupRate: 0.07,
    driftReduction: 0.1,
    consolidationBoost: 0.14
};
```

==========
FILE 18 — kernel/signalBus.js
==========

```
/* signalBus.js */
import { EventEmitter } from "events";
class SignalBus extends EventEmitter {}
export default SignalBus;
```

==========
FILE 19 — kernel/tickScheduler.js
==========

```
/* tickScheduler.js */
class TickScheduler {
    constructor(fps = 20) {
        this.fps = fps;
        this.interval = 1000 / fps;
        this.timer = null;
    }
    start(callback) {
        if (!this.timer) {
            this.timer = setInterval(callback, this.interval);
        }
    }
    stop() {
        if (this.timer) {
            clearInterval(this.timer);
            this.timer = null;
        }
    }
}
export default TickScheduler;
```

==========
FILE 20 — kernel/kernelEmitter.js
==========

```
/* kernelEmitter.js */
class KernelEmitter {
    emitState(state, bus) {
        bus.emit("KERNEL_DETAIL", state);
```

```
    }
}
export default new KernelEmitter();
```

==========
FILE 21 — kernel/kernelSensors.js
==========

```
/* kernelSensors.js */
class KernelSensors {
    constructor() {
        this.prev = null;
    }
    observe(state, bus) {
        if (!this.prev) this.prev = { ...state };
        const slope =
            state.memoryLoad - this.prev.memoryLoad ||
            state.resonanceValue - this.prev.resonanceValue;
        bus.emit("KERNEL_SLOPE", { slope });
        this.prev = { ...state };
    }
}
export default new KernelSensors();
```

==========
FILE 22 — genome/genomeCore.js
==========

```
/* genomeCore.js */
import GenomeStrands from "./genomeStrands.js";
import GenomeRegulators from "./genomeRegulators.js";
import GenomeExpression from "./genomeExpression.js";
import GenomeEmitter from "./genomeEmitter.js";
import GenomeSensors from "./genomeSensors.js";

class EmerickGenome {
    constructor(bus) {
        this.bus = bus;
        this.state = {
            activation: 1.0,
            expressionLevel: 0.5,
            plasticity: 0.5,
            modulationBias: 0.5,
            tick: 0
        };
        this.strands = GenomeStrands;
```

```
        this.regulators = GenomeRegulators;
        this.express = GenomeExpression;

        bus.on("DREAM_COMPLETE", () => {
            this.state.expressionLevel += 0.02;
        });
    }

    tick(modules) {
        this.state.tick++;
        this.express.apply(modules, this.state, this.strands, this.regulators);
        GenomeEmitter.emitState(this.state, this.bus);
        GenomeSensors.observe(this.state, this.bus);
    }
}

export default EmerickGenome;
```

==========
FILE 23 — genome/genomeStrands.js
==========

```
/* genomeStrands.js */
const GenomeStrands = {
    CHAOS_SHAPER: 0.68,
    AFFECT_GAIN: 0.74,
    RESONANCE_BIAS: 0.81,
    MEMORY_PLASTICITY: 0.93,
    DRIFT_CEILING: 0.57,
    DREAM_INTENSITY: 0.88,
    THOUGHT_INHIBITION: 0.49,
    DEVELOPMENT_SPEED: 0.62
};
export default GenomeStrands;
```

==========
FILE 24 — genome/genomeRegulators.js
==========

```
/* genomeRegulators.js */
const GenomeRegulators = {
    chaos: 0.7,
    emotion: 0.8,
    resonance: 0.6,
    memory: 1.0,
    lobes: 0.75,
```

```
    dream: 0.85,
    thought: 0.9
};
export default GenomeRegulators;
```

==========
FILE 25 — genome/genomeExpression.js
==========

```
/* genomeExpression.js */
class GenomeExpression {
    apply(modules, state, strands, regs) {
        const g = strands;
        const r = regs;
        const { chaos, emotion, resonance, memory, dream, thought } = modules;

        chaos.state.turbulence *= 1 + g.CHAOS_SHAPER * 0.05 * r.chaos;
        emotion.state.pressure *= 1 + g.AFFECT_GAIN * 0.03 * r.emotion;
        resonance.state.coherence *= 1 + g.RESONANCE_BIAS * 0.04 * r.resonance;
        memory.state.recallSignal *= 1 + g.MEMORY_PLASTICITY * 0.06 * r.memory;

        memory.nodes.forEach((n) => {
            n.strength = Math.min(1, n.strength + g.MEMORY_PLASTICITY * 0.005);
            n.drift = Math.min(g.DRIFT_CEILING, n.drift);
        });

        dream.state.intensity *= 1 + g.DREAM_INTENSITY * 0.1 * r.dream;

        if (thought.state.filtered) {
            thought.state.filtered.cognitiveTension *=
                1 - g.THOUGHT_INHIBITION * 0.02 * r.thought;
        }

        state.expressionLevel += g.DEVELOPMENT_SPEED * 0.001;
        state.plasticity = Math.min(1, state.expressionLevel);
    }
}
export default new GenomeExpression();
```

==========
FILE 26 — genome/genomeBiasMap.js
==========

```
/* genomeBiasMap.js */
class GenomeBiasMap {
    static applyLobeBias(value, bias) {
```

```
        return Math.min(1, value * bias);
    }
    static applyEmotionBias(value, gain, damping) {
        return value * gain * damping;
    }
    static applyChaosBias(noise, amp, smooth) {
        return noise * amp * smooth;
    }
}
export default GenomeBiasMap;
```

==========
FILE 27 — genome/genomeEmitter.js
==========

```
/* genomeEmitter.js */
class GenomeEmitter {
    emitState(state, bus) {
        bus.emit("GENOME_DETAIL", {
            activation: state.activation,
            expressionLevel: state.expressionLevel,
            plasticity: state.plasticity,
            modulationBias: state.modulationBias
        });
    }
}
export default new GenomeEmitter();
```

==========
FILE 28 — genome/genomeSensors.js
==========

```
/* genomeSensors.js */
class GenomeSensors {
    constructor() {
        this.prev = null;
    }
    observe(state, bus) {
        if (!this.prev) this.prev = { ...state };
        const slope = state.expressionLevel - this.prev.expressionLevel;
        bus.emit("GENOME_EXPRESSION_SLOPE", { slope });
        this.prev = { ...state };
    }
}
export default new GenomeSensors();
```

==========
FILE 29 — core/pulseEngine.js
==========

```
/* pulseEngine.js */
class PulseEngine {
    constructor(bus) {
        this.bus = bus;
        this.state = {
            bpm: 42,
            pulse: 0,
            tick: 0,
            oxygenDemand: 1.0,
            vascularPressure: 0.5
        };
    }
    tick(modules) {
        const { bpm } = this.state;
        this.state.tick++;
        this.state.pulse =
            Math.sin(this.state.tick * (bpm / 60) * 0.1);
        this.state.vascularPressure =
            0.5 + Math.abs(this.state.pulse) * 0.5;

        this.bus.emit("HEARTBEAT", {
            pulse: this.state.pulse,
            vascularPressure: this.state.vascularPressure
        });

        this.state.oxygenDemand =
            (modules.chaos.state.turbulence +
             modules.emotion.state.pressure +
             modules.resonance.state.coherence * 0.5) / 3;

        if (this.state.oxygenDemand > 1.2) this.state.bpm = Math.min(120, bpm + 1);
        else if (this.state.oxygenDemand < 0.8) this.state.bpm = Math.max(32, bpm - 1);
    }
}
export default PulseEngine;
```

==========
FILE 30 — core/oxygenationCycle.js
==========

```
/* oxygenationCycle.js */
class OxygenationCycle {
```

```
  constructor(bus) {
    this.bus = bus;
    this.state = { oxygenLevel: 1.0, saturation: 1.0, breathTick: 0 };
  }
  tick(modules) {
    const chaos = modules.chaos.state.turbulence;
    const emotion = modules.emotion.state.pressure;
    const resonance = modules.resonance.state.coherence;

    this.state.breathTick++;
    const stress = chaos * 0.4 + emotion * 0.4 - resonance * 0.3;

    this.state.oxygenLevel = 1 - Math.min(1, Math.max(0, stress));
    this.state.saturation =
      Math.sin(this.state.breathTick * 0.05) * 0.1 +
      this.state.oxygenLevel;

    modules.memory.state.recallSignal *= this.state.saturation;
    modules.thought.state.tensionRelease =
      this.state.saturation * 0.5;

    this.bus.emit("OXYGENATION", {
      oxygen: this.state.oxygenLevel,
      saturation: this.state.saturation
    });
  }
}
export default OxygenationCycle;
```

==========

FILE 31 — core/signalFlow.js

==========

```
/* signalFlow.js */
class SignalFlow {
  constructor(bus) {
    this.bus = bus;
    this.state = { flowRate: 0.5, nutrients: 1.0, waste: 0.0 };
  }

  tick(modules) {
    const tension = modules.thought.state.current?.cognitiveTension || 0;
    const dreamUsage = modules.dream.state.intensity || 0;

    this.state.flowRate = 0.3 + tension * 0.2 + dreamUsage * 0.1;
```

```
        this.state.nutrients = Math.max(0.1, this.state.nutrients - tension * 0.01);
        this.state.waste = this.state.waste + modules.memory.nodes.length * 0.0005;

        modules.frontal.state.energy = this.state.flowRate;
        modules.temporal.state.energy = this.state.flowRate;
        modules.limbic.state.energy = this.state.flowRate * 1.1;

        this.bus.emit("SIGNAL_FLOW", {
            flow: this.state.flowRate,
            nutrients: this.state.nutrients,
            waste: this.state.waste
        });
    }

    clearWaste() {
        this.state.waste *= 0.05;
    }
}

export default SignalFlow;
```

==========
FILE 32 — core/synapseBridge.js
==========

```
/* synapseBridge.js */
class SynapseBridge {
    constructor(bus) {
        this.bus = bus;
        this.weights = {
            frontal_to_thought: 0.7,
            temporal_to_memory: 0.9,
            parietal_to_resonance: 0.6,
            limbic_to_emotion: 0.85
        };
    }

    tick(modules) {
        const { frontal, temporal, parietal, limbic } = modules;

        modules.thought.state.inputBias =
            frontal.state.focus * this.weights.frontal_to_thought;

        modules.memory.state.strengthBias =
            temporal.state.patterns * this.weights.temporal_to_memory;
```

```
    modules.resonance.state.biasInfluence =
        parietal.state.model * this.weights.parietal_to_resonance;

    modules.emotion.state.limbicDrive =
        limbic.state.drive * this.weights.limbic_to_emotion;

    this.bus.emit("SYNAPSE_ACTIVITY", {
        ft: modules.thought.state.inputBias,
        tm: modules.memory.state.strengthBias,
        pr: modules.resonance.state.biasInfluence,
        le: modules.emotion.st
```

```
# luca_genome.py

import random
from dataclasses import dataclass

@dataclass
class GenomeVector:
    """
    The foundational genetic blueprint for LUCA.
    All behavior, metabolism, drift, and emotional gain originate from here.
    """

    plasticity_coefficient: float = 0.85   # Learning rate (higher = more drift)
    chaos_tolerance: float = 0.40          # Panic lock threshold
    emotional_gain: float = 1.2            # Affective intensity
    inhibition_bias: float = 0.30          # Cognitive filtering strength
    drift_ceiling: float = 0.70            # Entropy before dream purge triggers
    metabolic_efficiency: float = 0.90     # How fast oxygen burns

    def mutate(self, stress_factor: float):
        """
        Evolutionary drift. Internal genome mutation based on systemic stress.
        Higher stress = faster mutation.

        This function is extremely important — it produces long-term
        behavior changes across LUCA's "lifetime."
        """
        shift = (random.random() - 0.5) * stress_factor * 0.10
```

```
        # Plasticity mutation
        self.plasticity_coefficient = max(
            0.1, min(1.0, self.plasticity_coefficient + shift)
        )

        # Future versions can mutate more genes here…


# luca_physiology.py

from dataclasses import dataclass

@dataclass
class Physiology:
    """
    LUCA's metabolic body.

    This controls:
    - oxygenation (clarity of thought)
    - glucose (energy reserves)
    - fatigue (long-term exhaustion)
    - waste_entropy (mental noise accumulation)
    - heart_rate (internal clock influence)

    Behavior across the entire organism depends on this.
    """

    genome: any  # GenomeVector
    oxygenation: float = 1.0   # 1.0 = clear, 0.0 = hypoxic collapse
    glucose: float = 1.0       # metabolic energy
    fatigue: float = 0.0       # long-term exhaustion
    waste_entropy: float = 0.0 # “mental garbage” requiring dream purge
    heart_rate: int = 60       # influences tick dynamics

    def metabolize(self, cognitive_load: float):
        """
        Every thought burns oxygen, uses glucose, and generates entropy.

        The higher the input load:
        - the faster LUCA becomes confused
        - the faster hallucinations grow
        - the sooner a dream purge is required
        """
```

```
        burn_rate = cognitive_load * (1.0 / self.genome.metabolic_efficiency)

        # Burn resources
        self.oxygenation -= burn_rate * 0.01
        self.glucose -= burn_rate * 0.005
        self.waste_entropy += burn_rate * 0.02

        # Passive breathing recharge
        self.oxygenation += 0.005

        # Clamp ranges
        self.oxygenation = max(0.0, min(1.0, self.oxygenation))
        self.glucose = max(0.0, min(1.0, self.glucose))

        # Fatigue increases when waste builds up too high
        if self.waste_entropy > self.genome.drift_ceiling:
            self.fatigue = min(1.0, self.fatigue + 0.01)

    def get_performance_scalar(self) -> float:
        """
        Returns LUCA's short-term cognitive clarity.
        Low oxygenation = hallucination.
        High fatigue = inhibition of thought.

        1.0 = perfect clarity
        0.0 = total collapse
        """
        return self.oxygenation * (1.0 - self.fatigue)



# luca_chaos.py

class ChaosEngine:
    """
    LUCA’s chaotic subsystem.

    This is a Lorenz attractor — a nonlinear dynamical system.
    It ensures LUCA is never fully predictable, even when idle.

    The chaos value influences:
    - emotional arousal
    - memory drift
    - thought fragmentation
```

```
    - creativity spikes
    - panic susceptibility
    """

    def __init__(self):
        # Lorenz attractor state variables
        self.x = 0.1
        self.y = 0.1
        self.z = 0.1

        # Lorenz constants
        self.sigma = 10.0
        self.rho = 28.0
        self.beta = 8.0 / 3.0

        # simulation timestep
        self.dt = 0.01

    def tick(self) -> float:
        """
        Advance the chaos engine one timestep.
        Returns a normalized turbulence value between 0.0 and 1.0.
        """

        dx = (self.sigma * (self.y - self.x)) * self.dt
        dy = (self.x * (self.rho - self.z) - self.y) * self.dt
        dz = (self.x * self.y - self.beta * self.z) * self.dt

        # Update state
        self.x += dx
        self.y += dy
        self.z += dz

        # Rough normalization
        # Lorenz attractor wanders roughly between -20 and +20
        turbulence = (self.x + 20) / 40.0

        return max(0.0, min(1.0, turbulence))



# luca_emotion.py

class EmotionSwarm:
```

```
"""
LUCA's affective engine.

Produces:
- arousal (energy / alertness)
- valence (positive ↔ negative tone)
- dominance (control ↔ helplessness)
- panic spikes when chaos + low oxygen align
"""

def __init__(self, genome, physiology):
    self.genome = genome
    self.physiology = physiology

    self.arousal = 0.2
    self.valence = 0.0
    self.dominance = 0.5

def update(self, chaos_value: float):
    """
    Evolve emotional state based on metabolism + chaos.

    Emotion formula:
    arousal   ∝ chaos * emotional_gain
    valence   ∝ oxygenation - entropy
    dominance ∝ glucose + oxygenation
    """

    # AROUSAL: chaos-driven
    self.arousal += (
        chaos_value * 0.1 * self.genome.emotional_gain
        - self.physiology.fatigue * 0.05
    )

    # VALENCE: positivity vs negativity
    self.valence += (
        (self.physiology.oxygenation * 0.4) -
        (self.physiology.waste_entropy * 0.2)
    )

    # DOMINANCE: sense of control
    self.dominance += (
        (self.physiology.glucose * 0.3) +
        (self.physiology.oxygenation * 0.2) -
```

```
            (chaos_value * 0.1)
        )

        # Clamp everything
        self.arousal   = max(0.0, min(1.0, self.arousal))
        self.valence   = max(-1.0, min(1.0, self.valence))
        self.dominance = max(0.0, min(1.0, self.dominance))

    def is_panicking(self) -> bool:
        """
        Panic when:
        - chaos is high
        - oxygenation is low
        - dominance collapses

        This triggers runaway loops in LUCA's cognition.
        """

        return (
            self.arousal > 0.85 and
            self.physiology.oxygenation < 0.3 and
            self.dominance < 0.25
        )



# luca_memory.py

import random
import math

class MemoryMesh:
    """
    LUCA's memory system.

    This is a drifting, decaying, self-organizing associative network.
    Nodes strengthen with use and decay over time.
    Drift creates distortion, hallucination, and creativity.
    """

    def __init__(self, genome):
        self.genome = genome

        # internal memory: a list of { "vector": [...], "strength": float }
```

```
    self.nodes = []

    # drift rate derived from genome plasticity
    self.base_drift = 0.01 * self.genome.plasticity_coefficient

# ------------------------------
# RECORD NEW EXPERIENCE
# ------------------------------

def store(self, vector):
    """
    Store a new memory vector.
    Small random mutation is added because LUCA never records
    anything with perfect fidelity.
    """

    noisy_vector = [
        v + random.uniform(-0.01, 0.01)
        for v in vector
    ]

    self.nodes.append({
        "vector": noisy_vector,
        "strength": 1.0
    })

# ------------------------------
# MEMORY RECALL
# ------------------------------

def recall(self, cue_vector):
    """
    Recall the strongest matching memory node.

    Similarity = cosine similarity between vectors.
    """

    if not self.nodes:
        return None

    best_node = None
    best_score = -999.0

    for node in self.nodes:
```

```
        score = self._cosine_similarity(node["vector"], cue_vector)
        # weight by strength
        score *= node["strength"]

        if score > best_score:
            best_score = score
            best_node = node

    if best_node:
        # strengthen memory on recall
        best_node["strength"] = min(2.0, best_node["strength"] + 0.05)

        return best_node["vector"]

    return None

# ------------------------------
# MEMORY TICK (DRIFT + DECAY)
# ------------------------------

def tick(self, chaos_value):
    """
    Apply drift and decay to memory vectors.
    High chaos = more distortion.
    Low chaos = slow consolidation.
    """

    drift_rate = self.base_drift + (chaos_value * 0.02)

    for node in self.nodes:

        # Memory drift (hallucination)
        node["vector"] = [
            v + random.uniform(-drift_rate, drift_rate)
            for v in node["vector"]
        ]

        # Memory decay (forgetting)
        node["strength"] *= 0.995

    # purge weak memories
    self.nodes = [
        n for n in self.nodes
        if n["strength"] > 0.05
```

```
        ]

    # ------------------------------
    # COSINE SIMILARITY
    # ------------------------------

    def _cosine_similarity(self, a, b):
        dot = sum(x*y for x, y in zip(a, b))
        mag_a = math.sqrt(sum(x*x for x in a))
        mag_b = math.sqrt(sum(x*x for x in b))

        if mag_a == 0 or mag_b == 0:
            return 0.0

        return dot / (mag_a * mag_b)



# luca_cognition.py

import random
import math

class CognitionEngine:
    """
    LUCA's working cognitive space.

    This simulates:
    - short-term thought vectors
    - drift and fragmentation under stress
    - constraint filtering (inhibition)
    - emotional biasing of interpretation
    - hallucination proportional to chaos + low oxygen
    """

    def __init__(self, genome, physiology, chaos_engine, emotion_swarm, memory_mesh):
        self.genome = genome
        self.physiology = physiology
        self.chaos = chaos_engine
        self.emotion = emotion_swarm
        self.memory = memory_mesh

        # short-term working vector (like a tiny synthetic prefrontal cortex)
        self.working_vector = [0.0] * 8
```

```
# ----------------------------------------------------------
# THINKING STEP
# ----------------------------------------------------------

def think(self):
    """
    Main cognitive update.

    It performs:
    - chaos injection
    - metabolic performance scaling
    - emotional bias shifts
    - memory influence
    """

    chaos_value = self.chaos.tick()
    performance = self.physiology.get_performance_scalar()

    # Emotional and chaos-driven drift
    drift_strength = (
        chaos_value * 0.04 +
        (1.0 - performance) * 0.05 +
        self.emotion.arousal * 0.03
    )

    # Update each dimension of the working vector
    for i in range(len(self.working_vector)):
        # Core drift
        self.working_vector[i] += random.uniform(-drift_strength, drift_strength)

        # Constraint filtering (inhibition)
        self.working_vector[i] *= (1.0 - self.genome.inhibition_bias)

    # Bias thoughts by emotional valence
    self._apply_emotional_bias()

    # Memory influence (recall may shape thought)
    self._apply_memory_resonance()

    # Drifting thought naturally generates entropy
    self.physiology.waste_entropy += chaos_value * 0.01

    return self.working_vector
```

```
    # ----------------------------------------------------------
    # EMOTIONAL BIAS
    # ----------------------------------------------------------

    def _apply_emotional_bias(self):
        """
        When valence is negative, thought tilts toward unstable patterns.
        When positive, patterns stabilize.
        """

        bias = self.emotion.valence * 0.05

        for i in range(len(self.working_vector)):
            self.working_vector[i] += bias

    # ----------------------------------------------------------
    # MEMORY RESONANCE
    # ----------------------------------------------------------

    def _apply_memory_resonance(self):
        """
        Working vector is compared against memory mesh.
        Strong memory echoes may pull thought in their direction.
        """

        recalled = self.memory.recall(self.working_vector)

        if recalled:
            for i in range(len(self.working_vector)):
                self.working_vector[i] = (
                    self.working_vector[i] * 0.7 +
                    recalled[i] * 0.3
                )


# luca_teleology.py

class TeleologyCortex:
    """
    LUCA's goal-pressure engine.

    This simulates:
```

```
- survival priorities
- urgency
- pressure buildup under strain
- override impulses when stability collapses

NOT real motivation — a computational model of pressure dynamics.
"""

def __init__(self, physiology, emotion_swarm, genome):
    self.physiology = physiology
    self.emotion = emotion_swarm
    self.genome = genome

    # Core synthetic drives (0.0 - 1.0)
    self.stability_drive = 0.5      # maintain clarity + oxygen
    self.rest_drive = 0.3           # reduce fatigue
    self.novelty_drive = 0.4        # explore new patterns

    # Internal pressure accumulator
    self.pressure = 0.0

# ------------------------------------------------------------
# MAIN TELEOLOGY UPDATE
# ------------------------------------------------------------

def update(self):
    """
    Computes LUCA's internal synthetic 'goal pressure'.

    Pressure rises when:
    - oxygen is low
    - fatigue is high
    - entropy is high
    - emotions are extreme
    """

    # Pressure from metabolism
    self.pressure += (1.0 - self.physiology.oxygenation) * 0.05
    self.pressure += self.physiology.fatigue * 0.04
    self.pressure += self.physiology.waste_entropy * 0.03

    # Pressure from emotional turbulence
    if self.emotion.is_panicking():
        self.pressure += 0.10
```

```
        # Pressure from genome traits
        self.pressure *= self.genome.plasticity_coefficient

        # Clamp
        self.pressure = max(0.0, min(1.0, self.pressure))

        return self.pressure

    # ------------------------------------------------------------
    # DERIVED "GOAL" SUGGESTIONS (NOT REAL GOALS)
    # ------------------------------------------------------------

    def get_action_bias(self):
        """
        Generates synthetic action impulses.

        These are NOT actual intentions — just computational hints
        that influence how LUCA's cognition will drift.
        """

        pressure = self.pressure

        return {
            "seek_stability": self.stability_drive * pressure,
            "seek_rest": self.rest_drive * pressure,
            "seek_novelty": self.novelty_drive * (1.0 - pressure)
        }

    # ------------------------------------------------------------
    # PRESSURE OVERRIDE EVENT
    # ------------------------------------------------------------

    def should_override_constraints(self):
        """
        When pressure reaches a severe threshold,
        the system signals the need to break from normal patterns.
        """

        return self.pressure > 0.85



# luca_core.py
```

```
import time

from luca_genome import GenomeVector
from luca_physiology import Physiology
from luca_chaos import ChaosEngine
from luca_emotion import EmotionSwarm
from luca_memory import MemoryMesh
from luca_cognition import CognitionEngine
from luca_teleology import TeleologyCortex


class LUCA:
    """
    LUCA Core Kernel
    ----------------
    This class ties all subsystems together into a coherent
    synthetic organism simulation.

    This is software — a computational toy model of a biological system.
    """

    def __init__(self, tick_rate_hz: float = 20.0):
        self.tick_rate = tick_rate_hz
        self.dt = 1.0 / self.tick_rate

        # Instantiate genome first
        self.genome = GenomeVector()

        # Instantiate subsystems
        self.physiology = Physiology(self.genome)
        self.chaos = ChaosEngine()
        self.emotion = EmotionSwarm(self.genome, self.physiology)
        self.memory = MemoryMesh(self.genome)
        self.cognition = CognitionEngine(
            self.genome, self.physiology, self.chaos, self.emotion, self.memory
        )
        self.teleology = TeleologyCortex(self.physiology, self.emotion, self.genome)

        # Internal counters
        self.tick_count = 0

    # ------------------------------------------------------------
    # MAIN TICK
```

```
# ----------------------------------------------------------

def tick(self):
    """
    One simulation step:
    - chaos engine updates
    - cognition runs
    - emotion evolves
    - teleology pressure updated
    - memory drift
    - metabolism update
    """

    chaos_value = self.chaos.tick()

    # cognition produces working thought vector
    thought = self.cognition.think()

    # emotion evolves based on turbulence
    self.emotion.update(chaos_value)

    # teleology generates pressure and impulses
    pressure = self.teleology.update()
    biases = self.teleology.get_action_bias()

    # memory mesh drifts every tick
    self.memory.tick(chaos_value)

    # metabolism reacts to cognitive load
    cognitive_load = sum(abs(x) for x in thought) / len(thought)
    self.physiology.metabolize(cognitive_load)

    self.tick_count += 1

    # return a snapshot for UI/logging
    return {
        "tick": self.tick_count,
        "thought": thought,
        "arousal": self.emotion.arousal,
        "valence": self.emotion.valence,
        "dominance": self.emotion.dominance,
        "pressure": pressure,
        "biases": biases,
        "oxygen": self.physiology.oxygenation,
```

```
            "glucose": self.physiology.glucose,
            "entropy": self.physiology.waste_entropy,
            "fatigue": self.physiology.fatigue,
            "chaos": chaos_value,
        }

    # ------------------------------------------------------------
    # RUN LOOP
    # ------------------------------------------------------------

    def run(self, ticks: int = 100):
        """
        Run LUCA for a set number of ticks.

        You can replace this with an async loop, threaded loop,
        or front-end socket later.
        """
        for _ in range(ticks):
            snapshot = self.tick()
            time.sleep(self.dt)
            yield snapshot



# run_luca.py

from luca_core import LUCA

def main():
    """
    Simple execution script for the LUCA synthetic-organism simulation.
    This is software only — a computational model.
    """

    luca = LUCA(tick_rate_hz=10.0)  # slower so you can read outputs

    print("Starting LUCA simulation...\n")

    # Run 50 ticks (you can change this number)
    for snapshot in luca.run(ticks=50):
        print(
            f"Tick: {snapshot['tick']} | "
            f"Chaos: {snapshot['chaos']:.2f} | "
            f"O2: {snapshot['oxygen']:.2f} | "
```

```
            f"Entropy: {snapshot['entropy']:.2f} | "
            f"Arousal: {snapshot['arousal']:.2f} | "
            f"Pressure: {snapshot['pressure']:.2f}"
        )

    print("\nSimulation complete.")

if __name__ == '__main__':
    main()



# luca_monitor.py

import time
import os
from luca_core import LUCA

class LUCAMonitor:
    """
    Console-based monitor for the LUCA simulation.
    Displays internal physiological and cognitive values
    in a readable, real-time dashboard.

    Purely a viewer — it does not influence LUCA.
    """

    def __init__(self, tick_rate_hz=10):
        self.luca = LUCA(tick_rate_hz=tick_rate_hz)

    def clear(self):
        # Clear screen (Windows or Unix)
        os.system('cls' if os.name == 'nt' else 'clear')

    def draw_panel(self, snapshot):
        """
        Draws a simple dashboard to the terminal.
        """

        print("=== LUCA SOFTWARE SIMULATION — MONITOR ===\n")

        print(f"Tick:         {snapshot['tick']}")
        print(f"Chaos:        {snapshot['chaos']:.2f}")
        print(f"Oxygenation:  {snapshot['oxygen']:.2f}")
```

```
        print(f"Glucose:      {snapshot['glucose']:.2f}")
        print(f"Entropy:      {snapshot['entropy']:.2f}")
        print(f"Fatigue:      {snapshot['fatigue']:.2f}\n")

        print(f"Arousal:      {snapshot['arousal']:.2f}")
        print(f"Valence:      {snapshot['valence']:.2f}")
        print(f"Dominance:    {snapshot['dominance']:.2f}")
        print(f"Pressure:     {snapshot['pressure']:.2f}\n")

        print(f"Bias → Stability: {snapshot['biases']['seek_stability']:.2f}")
        print(f"Bias → Rest:      {snapshot['biases']['seek_rest']:.2f}")
        print(f"Bias → Novelty:   {snapshot['biases']['seek_novelty']:.2f}\n")

        print("Working Vector:")
        v = snapshot['thought']
        formatted = "[" + ", ".join(f"{x:.2f}" for x in v) + "]"
        print(f"  {formatted}\n")

        print("(Press CTRL+C to stop simulation)")

    def run(self, ticks=None):
        """
        Run the monitor in real-time.
        If ticks=None, runs indefinitely.
        """

        print("Launching LUCA monitor...")
        time.sleep(1)

        tick_limit = ticks if ticks is not None else float('inf')

        for _ in range(int(tick_limit)):
            snapshot = self.luca.tick()

            self.clear()
            self.draw_panel(snapshot)

            time.sleep(self.luca.dt)


def main():
    monitor = LUCAMonitor(tick_rate_hz=10)
    monitor.run()  # runs until you CTRL+C
```

```
if __name__ == "__main__":
    main()
```